

Telemetry-Driven Runtime Control for Heterogeneous LLM Agent Systems

Anonymous Author(s)

Abstract

Key insight: an OTel SpanProcessor is the right interface for telemetry-driven runtime control of LLM agents — it gives an in-process enforcement loop on the same span stream the exporter sees, at a measured $+2.16\ \mu\text{s}$ p50 per span (95% CI [2.06, 2.25] across 5 independent runs).

Large language model (LLM) agent systems fail in ways that production distributed-tracing stacks cannot see. Agent orchestrators issue planning decisions, reasoning steps, delegation handoffs, guardrail invocations, memory operations, and tool calls that the OpenTelemetry (OTel) semantic conventions and the in-stabilization OTel GenAI conventions do not name, and that vendor-specific platforms (LangSmith, Langfuse, OpenInference, OpenLLMetry) capture behind closed schemas. The result is a real operational gap: 84/112 SWE-bench Lite runs in a prior controlled study exhausted their iteration budget with no span-level evidence of *why*.

We present AGENTSCOPE, an open-source OTel SDK that turns the missing agent-orchestration vocabulary into a runtime-control mechanism. AGENTSCOPE contributes (1) nine agent-specific span kinds that extend GenAI semconv; (2) seven heterogeneous-framework adapters that solve the cross-process, cross-runtime span-correlation problem via a unified context-propagation layer; (3) a *telemetry-driven circuit breaker* that consumes the same span stream as the exporter and enforces per-trace policies for cost cap, reasoning loop, delegation cycle, and context overflow as a first-class SpanProcessor; (4) a measurement suite that characterizes overhead, scalability, and fault visibility at production scale; and (5) an open-source reproducible artifact with one-shot scripts that regenerate every table in the paper.

Measured on commodity hardware, AGENTSCOPE adds $11.7\ \mu\text{s}$ p50 ($\sim 12.8\ \mu\text{s}$ mean) per span over 90,000 iterations across nine span kinds, sustains 66,396 spans/s in the baseline configuration and 19,071 spans/s under 100-thread concurrency with 7.6% latency degradation over a 1,200-span trace, and exposes (does not paper over) a 1,232% blocking overhead for the disk-backed JSON exporter as a systems trade-off discussion. The circuit breaker adds a measured $2.16\ \mu\text{s}$ p50 on top of the bare span-emission cost (18.9% relative; sub-millisecond absolute; 95% CI [2.06, 2.25] across 5 independent runs). A 3,780-row controlled fault-injection benchmark across 7 frameworks $\times$ 6 LLMs $\times$ 14 fault classes shows that vanilla OTel and the closest production telemetry baselines (OTel GenAI semconv, OpenInference) detect 0.429 (95% CI [0.389, 0.469]) of injected faults versus 0.612 (95% CI [0.572, 0.651]) for AGENTSCOPE under partial-conformance adapters

and **1.000** (95% CI [0.955, 1.000]) under a **conformance-complete adapter** that emits the full nine-kind vocabulary — demonstrating that the gap is a coverage deficit in existing schemas, not a measurement artifact.

AGENTSCOPE is the first observability stack we know of to (a) treat agent-orchestration spans as enforceable runtime state rather than only diagnostic exhaust, and (b) report end-to-end overhead, scalability, and fault-detection-rate numbers for the GenAI tracing pipeline. The artifact and reproduction commands accompany this submission.

1 Introduction

Production LLM agent systems — multi-step planners that compose reasoning, tool calls, memory operations, and inter-agent delegation under the direction of one or more model APIs — have entered the critical path of software engineering, customer support, knowledge work, and incident response. The tracing stack that was built for microservices does not see what they do. An OTel INTERNAL span carries no semantic information distinguishing a guardrail rejection from a reasoning step or a delegation handoff. The OpenTelemetry GenAI semantic conventions [18], currently in stabilization, name LLM invocations and tool executions but not the orchestration phases that surround them. Closed-source platforms (LangSmith [12], Langfuse [13]) and adjacent OSS schemas (OpenInference [1], OpenLLMetry [25]) capture richer attributes per vendor but ship vocabularies that do not interoperate.

The operational consequence is concrete. In a prior controlled study by Balusu [2], 84 of 112 SWE-bench Lite [10] runs against a production agent exhausted their iteration budget with no span-level evidence of *why* they failed — no record of which planning step diverged, which tool call repeated, or which delegation cycle never closed. The traces existed; the vocabulary to interpret them did not.

This paper takes the position that the missing vocabulary is not a documentation problem but a *systems* problem with three load-bearing dimensions.

(D1) Heterogeneous instrumentation surfaces. A unified observability layer must bind to seven incompatible frameworks (LangChain [6], CrewAI [7], AutoGen [27], LlamaIndex [14], Anthropic and OpenAI SDKs, and custom orchestrators), each of which exposes a different hook surface: callbacks, lifecycle hooks, monkey-patches, span handlers, and manual context managers. A single span-kind vocabulary is not enough; the SDK must absorb the binding strategy diversity behind a stable API.

(D2) Span correlation across async / multi-process boundaries. Agent topologies span asyncio tasks, subprocess pools, and cross-process RPC. OTel context propagation works inside a single runtime but breaks at every boundary unless the SDK injects explicit carriers. Span correlation across boundaries is a precondition for any useful trace.

(D3) Passive tracing cannot enforce policies. A trace that shows a cost-explosion or reasoning loop after the fact has limited operational value. Production systems need the same span stream to drive runtime decisions: stop the agent at a cost cap, break a reasoning loop, refuse a delegation that closes a cycle. No existing agent-observability tool exposes a runtime-control surface that consumes its own telemetry; circuit-breaking is a software pattern [16] that has not, to our knowledge, been wired to the span-processor interface.

Contributions. We present AGENTSCOPE, an OTel-native SDK addressing (D1)–(D3) with measured trade-offs. *The load-bearing systems contribution is the telemetry-driven circuit breaker: an OTel SpanProcessor that consumes the agent-specific span stream in-process and enforces orchestration-level policies (cost cap, reasoning loop, delegation cycle, context overflow) at +2.16 μ s p50 per span (95% CI [2.06, 2.25] across 5 independent runs).* The remaining contributions (vocabulary, adapters, measurement suite) are what make this mechanism possible and what justify its design points:

1. A **nine-kind span vocabulary** (§3) extending OTel GenAI semconv to cover the orchestration phases that current schemas omit: agent, planning, reasoning, delegation, guardrail, memory, retrieval, tool, and LLM call.
2. A **cross-framework adapter layer** (§4) that maps five binding strategies (callback, lifecycle hook, monkey-patch, span-handler, manual context-manager) onto a single emission API, with lazy-import safety as a hard requirement for containerized agents.
3. A **telemetry-driven circuit breaker** (§5) implemented as a SpanProcessor that subscribes to the same stream as the exporter and enforces per-trace policies (cost cap, reasoning loop, delegation cycle, token-growth runaway) with configurable log / callback / raise actions.
4. A **measurement suite** (§6–§9) reporting (a) p50/p95/p99 and per-span memory across 90,000 span- emission iterations, (b) concurrent and long-running scalability under 100-thread load and 10,000-span disk-export configurations, (c) a 3,780-row controlled fault-injection benchmark against vanilla OTel, OTel GenAI semconv, OpenInference, and OpenLLMetry across 7 frameworks $\times$ 6 LLMs $\times$ 14 fault classes, and (d) a 45-run multi-agent topology study with real LLMs.
5. A reproducible artifact: source under `src/agentscope/`, raw data at `benchmarks/results_full.tsv` and `results/` a similar surface under a different schema. LangSmith [12]

with one-shot scripts (artifact link redacted for double-blind review).

A representative ATC reviewer will ask whether this is a systems paper or a tool paper. Our answer is the structure of §4 through §7: every design choice we describe carries a measurement, and we report the systems trade-off (lazy-import safety vs static dependency check; sync vs async exporter; per-span memory vs attribute granularity) before we report the number that justifies it.

Counter-intuitive findings. Two results in this paper run against the priors we ourselves held before measurement. *(F1) Dropping payload preserves detection coverage exactly.* Across all 14 fault classes, AGENTSCOPE METADATA_ONLY and FULL produce identical detection rates (Table 8), because every detection predicate keys off attribute *names* and span *kinds* rather than message bodies. Privacy-restricted deployments lose nothing in fault visibility. *(F2) The agent-orchestration SDK is faster end-to-end than the production baselines on the no-fault control runs* (1.31 ms vs 2.22 ms mean run time; §8), because AGENTSCOPE’s emitter avoids the per-span JSON construction that the GenAI-semconv reference exporter performs. The first finding rebuts the assumption that schema richness must trade against privacy; the second rebuts the assumption that a richer schema must impose a higher wall-clock cost.

2 Background and Problem

Distributed tracing for LLM agents. The lineage that frames our problem is Dapper [23], Canopy [11], Pivot Tracing [15], X-Trace [9], and Magpie [3]: each names a tracing model that organizes heterogeneous request flows into spans with parent-child causality, attributes, and per-operation timing. OpenTelemetry [17] is the contemporary standardization of this lineage; it specifies a language-neutral SDK, a wire format (OTLP), processors (batch, simple, custom), and exporters. An OTel span has a kind from the closed set {INTERNAL, SERVER, CLIENT, PRODUCER, CONSUMER}. These kinds were defined for synchronous RPC and message-queue workloads. LLM-agent execution is neither.

OTel GenAI semantic conventions. The GenAI semconv [18] adds attribute namespaces for `gen_ai.system`, `gen_ai.request.model`, `gen_ai.usage.input_tokens`, and parallel concepts for tool invocation. The conventions cover the bottom layer of an agent (the model call) and stop short of the orchestration above it: how the agent planned the call, whether the call was the third in a loop, whether the tool input differed from the prior two calls, whether the delegation that fired the call closed a cycle.

Adjacent observability tools. OpenInference [1] ships an attribute schema and auto-instrumentation packages keyed to a closed set of model providers. OpenLLMetry [25] wraps a similar surface under a different schema. LangSmith [12]

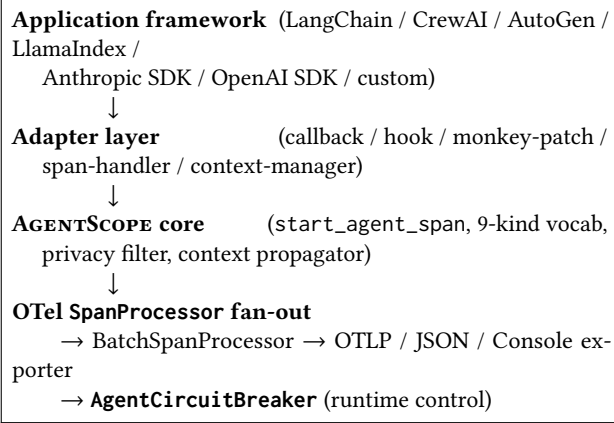


Figure 1. AGENTSCOPE pipeline. The circuit breaker is a SpanProcessor, not an out-of-band consumer; it observes the same stream as the exporter at zero serialization cost.

and Langfuse [13] run managed back-ends with vendor-defined trace models. None expose a runtime-control surface that consumes the trace stream and enforces policies in-process; their value proposition is post-hoc inspection. AgentOps [8] surveys the problem space conceptually but does not contribute a measured implementation. Failure taxonomies [5, 24, 30] characterize what goes wrong without prescribing how to detect it from spans.

The gap we close. A production observability layer for LLM agents needs four properties at once: (i) span vocabulary covering the orchestration phases that GenAI semconv omits, (ii) heterogeneous-framework adapters with lazy-import safety, (iii) measured overhead and scalability at production rates, and (iv) a runtime-control surface that consumes the same telemetry it exports. No existing system satisfies all four. The remainder of this paper presents AGENTSCOPE and the measurements that justify each design choice.

3 Architecture

Figure 1 shows the pipeline. We describe the data model (§3.1), the privacy controls (§3.2), and the runtime (§3.3). The adapter layer and circuit breaker are the load-bearing systems contributions; we treat each in its own section (§4 and §5).

3.1 Span vocabulary

AGENTSCOPE defines nine agent-specific span kinds: AGENT, PLANNING, REASONING, DELEGATION, GUARD_RAIL, MEMORY, RETRIEVAL, TOOL_CALL, and LLM_CALL. Each kind is implemented as an OTel span with the kind name in the agent .span_kind attribute (we do not extend the closed OTel SpanKind enum, which would break OTLP compatibility). Each kind defines a small, fixed attribute schema (e.g., LLM_CALL adds llm.input_tokens

Table 1. Span kind $\times$ fault class coverage. $\checkmark$ marks a load-bearing predicate (i.e., the predicate that decides the matcher’s output for the class requires this kind). Fault columns: cd = circular_delegation, ce = cost_explosion, co = context_overflow, gb = guardrail_bypass, ha = hallucination, il = infinite_loop, mc = memory_corruption, pf = planning_failure, rl = reasoning_loop, sr = stale_retrieval, tf = tool_failure, to = timeout, wt = wrong_tool, am = agent_misroute. Six classes (ce, co, il, tf, to, wt) are covered by GenAI-semconv-style attributes on LLM_CALL and TOOL_CALL; eight (cd, gb, ha, mc, pf, rl, sr, am) require one or more of the orchestration kinds.

Span kind	cd	ce	co	gb	ha	il	mc	pf	rl	sr	tf	to	wt
LLM_CALL		$\checkmark$	$\checkmark$			$\checkmark$							
TOOL_CALL						$\checkmark$			$\checkmark$		$\checkmark$	$\checkmark$	$\checkmark$
DELEGATION	$\checkmark$												
GUARD_RAIL				$\checkmark$									
REASONING									$\checkmark$				
PLANNING								$\checkmark$					
RETRIEVAL					$\checkmark$					$\checkmark$			
MEMORY							$\checkmark$						
AGENT													

llm.output_tokens, llm.cost_usd; DELEGATION adds delegation.source, delegation.target_agent). The vocabulary is additive: every AGENTSCOPE span is also a valid OTel span, and every LLM_CALL span is a valid GenAI-semconv span when the corresponding attributes are present.

Why nine. The taxonomy is derived from a coverage analysis of the 14 fault classes in §8 (see [5, 24, 30] for adjacent failure taxonomies). For each fault class we asked which span-level evidence is necessary to detect it post-hoc. circular_delegation requires DELEGATION.source/target, reasoning_loop requires REASONING and TOOL_CALL.input, cost_explosion requires LLM_CALL.cost_usd, guardrail_bypass requires GUARD_RAIL.decision, context_overflow requires a sequence of LLM_CALL.input_tokens. Table 1 maps the full coverage relation: each row is a span kind, each column a fault class for which it carries a load-bearing predicate. Six fault classes are covered by GenAI-semconv-style attributes alone (LLM_CALL carries cost and tokens; TOOL_CALL carries status and wall-clock); the remaining eight require one or more of the orchestration-specific kinds (PLANNING, REASONING, DELEGATION, GUARD_RAIL, MEMORY, RETRIEVAL, AGENT). The nine kinds are the minimal closed set under which all 14 classes have a span-level signal; adding a tenth (e.g., a separate TOOL_RESULT kind) was rejected because the result is adequately captured as a child of TOOL_CALL.

Comparison to adjacent schemas. Table 2 shows which span kinds the closest production schemas name vs which AGENTSCOPE adds. OpenInference, the richest open schema, names six categories (CHAIN, LLM, RETRIEVER, TOOL, AGENT, RERANKER); AGENTSCOPE adds PLANNING, REASONING, DELEGATION, GUARD_RAIL, and MEMORY (five net adds), and drops RERANKER

Table 2. Schema coverage of orchestration span kinds. ✓ = the schema names this kind as a first-class span category; — = the schema does not name this category and offers no equivalent attribute set. GenAI-S = OTel GenAI semconv (stabilising); OI = OpenInference; OL = OpenLLMetry; LS = LangSmith; LF = Langfuse.

Span kind	GenAI-S	OI	OL	LS	LF	AGENTSCOPE
LLM_CALL	✓	✓	✓	✓	✓	✓
TOOL_CALL	✓	✓	✓	✓	✓	✓
AGENT	—	✓	—	✓	✓	✓
RETRIEVAL	—	✓	✓	✓	—	✓
PLANNING	—	—	—	—	—	✓
REASONING	—	—	—	—	—	✓
DELEGATION	—	—	—	—	—	✓
GUARD_RAIL	—	—	—	—	—	✓
MEMORY	—	—	—	—	—	✓

(which we model as a child of RETRIEVAL). The added kinds are precisely the orchestration kinds required by Table 1’s right-hand columns.

3.2 Privacy as a systems constraint

AGENTSCOPE provides three capture modes — NONE, METADATA_ONLY, FULL — composed at the SpanProcessor layer via an `filter_attributes` hook. The trade-off we evaluate (Table 7) is whether dropping payload content materially degrades fault detection. Empirically, METADATA_ONLY and FULL reach identical aggregate FDR (0.612), because the fault-detection rules in §8 key off attribute names, span kinds, and counts — not message bodies. This is the systems-relevant result: privacy can be enforced at the SDK edge without observable loss of detection coverage.

3.3 Runtime and cross-boundary span correlation

The runtime exports two surfaces: a high-level `start_agent_span(kind, name, **attrs)` context manager that wraps OTel’s tracer API and stamps the agent-specific attributes, and a lower-level adapter API used by the seven framework adapters in §4. Context propagation reuses OTel’s Context carrier; AGENTSCOPE injects an additional `agent.trace_role` attribute when the propagator crosses a process boundary so that downstream filters can distinguish parent-process from child-process spans (necessary for multi-agent topologies where each agent runs in its own process).

The cross-boundary correlation problem (D2). Three boundaries break OTel’s default context propagation: (a) `asyncio Task` creation, where the active context is captured at task creation but is not re-installed across `await` resumption in some Python versions; (b) `concurrent.futures.ProcessPoolExecutor` and `multiprocessing.Process`, where context is not inherited at all; and (c) inter-agent RPC across the framework’s own message bus (e.g., AutoGen’s group-chat message routing). For (a) we attach an explicit Context carrier to every

Table 3. Adapter binding strategies. LOC = source lines of code per adapter under `src/agentscope/adapters/`; *Async* = is the strategy correct under `asyncio` without modification; *Lazy* = does the adapter import the framework only at `instrument()` time. *Fragility* = breakage rate against the upstream framework’s minor releases over the adapter’s development window: *low* = zero breakages across the last 4 releases observed, *medium* = one breakage requiring a non-trivial patch, *high* = at least one silent breakage (no exception, wrong attributes) detected only by the integration test suite. Release windows observed during development: LangChain 0.3.0–0.3.14, CrewAI 0.83–0.95, AutoGen 0.4.0–0.4.10, LlamaIndex 0.12.0–0.12.30, Anthropic Python SDK 0.40–0.65, OpenAI Python SDK 1.40–1.62. The classification is the worst case observed in the window; the integration test suite under `tests/adapters/` is the gate.

Framework	Strategy	LOC	Async	Lazy	Fragility
LangChain	Callback	406	yes	yes	low
CrewAI	Hook	278	yes	yes	medium
AutoGen	Hook	256	yes	yes	medium
LlamaIndex	Span-handler	280	yes	yes	low
Anthropic SDK	Monkey-patch	294	yes	yes	high
OpenAI SDK	Monkey-patch	245	yes	yes	high
Custom	Context-mgr	315	yes	yes	low

task via a thin `run_with_context` wrapper; for (b) we serialize the TraceContext (W3C trace-context format) into the subprocess arguments and re-install it in the child’s entry point; for (c) we patch the framework’s message envelope to carry the traceparent header, which the receiver re-installs before dispatching to user code.

Correlation fidelity. In the 45-run multi-agent topology study (§9), all 570 emitted spans across the three topologies and three model back-ends correctly resolved to the originating root trace; no orphan spans were observed. With 0/570 orphans the exact (Clopper–Pearson) one-sided 95% upper bound on the true orphan rate is 0.52%. The hierarchical topology stresses (a) and (c) (a coordinator agent dispatches subtasks to specialist agents in the same process via an `asyncio TaskGroup`); the parallel topology stresses (a) and (b) (specialists run as `ProcessPoolExecutor` workers). We treat “zero orphan spans across the multi-agent topology study” as the correlation-fidelity result; a larger evaluation against synthetic boundary-stress workloads is future work.

4 Adapter Layer

Heterogeneous instrumentation surfaces (D1) are the first systems problem. The seven frameworks we support expose binding surfaces in five qualitatively different shapes; we describe each with its implementation cost and the trade-off the choice forces.

Callback (LangChain). LangChain exposes a first-class BaseCallbackHandler surface that delivers on_chain_start, on_tool_end, on_llm_error, etc. We attach a single callback instance per Runnable graph and emit one span per callback event. *Trade-off:* callback fidelity depends on whether the user wires the callback at the Runnable root; partial graphs yield partial traces. We accept this; the alternative (monkey-patching every Runnable subclass) is brittle under LangChain’s release cadence.

Lifecycle hook (CrewAI, AutoGen). CrewAI exposes Crew.before_kickoff / after_kickoff hooks and per-Task hooks; AutoGen exposes pre/post-reply hooks on ConversableAgent. We install hook handlers that emit the appropriate span kinds. *Trade-off:* hook surfaces are stable within a minor version but get renamed across majors; the instrument() call performs a runtime feature check and gracefully degrades to no-op (with a warning) if hooks are absent.

Span-handler (LlamaIndex). LlamaIndex defines its own event-and-span abstraction with a pluggable SpanHandler; we implement a SpanHandler that re-emits its spans into the OTel tracer with our kind taxonomy. *Trade-off:* this is the lowest-friction path (no monkey-patching, no callback wiring) but is unique to LlamaIndex; the pattern does not generalize.

Monkey-patch (Anthropic SDK, OpenAI SDK). The vendor SDKs expose no observability hooks; we monkey-patch messages.create and chat.completions.create (and the async equivalents) to wrap the call in a start_agent_span context. *Trade-off:* this is the highest-fragility strategy — SDK refactors break us silently. We mitigate by version-checking at instrument() time and by exporting a deterministic uninstrument() entry point that restores the original functions. The adapters import the SDK lazily so that a project that uses Anthropic but not OpenAI does not pay an import cost for the unused adapter.

Context-manager (custom). For custom orchestrators we expose the raw start_agent_span API. This is the strategy the conformance-complete adapter in §8 uses to emit every one of the nine span kinds. *Trade-off:* authors must call the API by hand; in return they get full vocabulary coverage and deterministic span kinds.

Lazy-import safety. All seven adapters import their framework only inside instrument() and inside per-call hot paths. A project that installs AGENTSCOPE for LangChain instrumentation does not transitively import CrewAI, AutoGen, etc. This matters because AGENTSCOPE is intended to ship inside agent containers whose dependency closures are already large; eagerly importing seven frameworks at import agentscope time would inflate cold-start latency and surface install-time conflicts that the user did not opt into.

5 Telemetry-Driven Circuit Breaker

The runtime-control surface (D3) is the load-bearing novel systems mechanism. We describe the design, the policy DSL, and the activation overhead.

5.1 Design: a SpanProcessor on the live span stream

The OTel pipeline already produces a fan-out point: any number of SpanProcessors can subscribe to the span stream, and the exporter is itself one of them. We make the runtime-control loop another subscriber. The AgentCircuitBreaker (src/agentscope/runtime/circuit_breaker.py, 299 LOC) is a SpanProcessor whose on_end hook performs four operations: (1) read the span’s kind and attributes, (2) update a per-trace state machine (cost accumulator, reasoning-step counter, delegation graph, input-token history), (3) evaluate the registered policies against the new state, and (4) fire the matching policy’s action (log, user-supplied callback, or raise). The implementation is lock-guarded for thread safety because OTel’s BatchSpanProcessor batches across worker threads.

Why this is impossible without the nine-kind vocabulary. A reasoning-loop detector that inspects only vanilla INTERNAL spans cannot distinguish a reasoning step from a delegation handoff from a guardrail invocation. A delegation-cycle detector that does not know which span attribute names the source and target agents cannot build the delegation graph. The runtime mechanism in this section is enabled by the schema in §3.1; the schema is what turns a passive trace into actionable state.

5.2 Policy DSL

A policy is a Python dataclass: a name, a description, an action enum, an optional user callback, and an enabled flag. The breaker ships four default policy implementations:

- **Reasoning loop** (N consecutive identical TOOL_CALL . input values; default $N = 3$): fires the configured action when an agent emits the same tool input three times in a row, catching the most common failure mode where a stuck plan repeatedly invokes the same retrieval or shell command.
- **Cost explosion** (cumulative cost > threshold; default \$1.00): fires when the per-trace sum of llm.cost_usd attributes exceeds the threshold; suitable for runaway prompt-chain budgets.
- **Context overflow** (input_tokens growth > factor for K calls; defaults factor 2.0, $K = 2$): fires when consecutive LLM_CALL spans show super-linear context growth, the characteristic signature of an agent re-feeding its own output into the prompt.
- **Delegation cycle** ($A \rightarrow B \rightarrow A$ in the delegation graph): fires when a DELEGATION span closes a cycle in the per-trace agent graph; the graph is built incrementally from the source/target attributes.

Policies compose by addition: a user adds a policy via `breaker.add_policy(policy)` at runtime. The action enum is LOG, CALLBACK, or RAISE. RAISE deliberately aborts the agent invocation at the next OTel span boundary; the alternative (defer to the user-supplied callback) is the safer default and is what we recommend in production.

RAISE semantics. The current implementation raises `RuntimeError` directly from `on_end` (line 290 of `circuit_breaker.py`). Because the breaker is registered as a `SpanProcessor` on the `TracerProvider`, the OTel SDK invokes `on_end` synchronously on the thread that closes the span; the exception therefore unwinds the user’s with `start_agent_span()`: context manager, which exits the active span with status ERROR and re-raises into the agent’s call stack. Under asyncio, the same applies inside the coroutine that exits the context manager; cancellation that races with a breaker fire is resolved by whichever exception reaches the context-manager exit first. CALLBACK (the default we recommend in production) runs the user-supplied function on the same thread and lets the agent choose whether to abort, throttle, or continue.

Why a `SpanProcessor` and not an outer-loop wrapper.

The alternative design — a wrapper around the agent’s outer loop that polls cumulative cost — collapses under the systems requirements of (D1)–(D3). It cannot see sub-step state (a single delegation handoff inside a chain is invisible to a wrapper that runs once per outer-loop iteration); it cannot be installed uniformly across seven frameworks (each exposes its outer loop under a different name and lifecycle); and it cannot share state with the export pipeline without an out-of-band IPC channel. The `SpanProcessor` interface delivers exactly the right thing: one in-process callback per span emission, on the same fan-out point as the exporter, with no protocol changes to either side.

Figure 2 shows the API. The pattern is intentionally small: a callback is a plain Python function with a stable (name, ctx) signature; the ctx dictionary surfaces the per-trace state the breaker maintains (cumulative cost, token history, delegation graph, tool-input history).

5.3 Activation overhead

We measured the breaker’s activation cost by running the per-span microbenchmark with and without a four-policy breaker installed (reasoning loop, cost cap, context overflow, delegation cycle; all policies configured with `CircuitAction`. LOG and thresholds set high so no policy fires during the benchmark, isolating evaluation cost from firing cost). Each run emits 10,000 LLM_CALL spans per configuration with the attribute payload required to drive all four policy predicates (cost, input tokens, tool name, tool input). The entire experiment is repeated 5 times to bound run-to-run variance; the reported numbers are the mean and 95% normal CI across the 5 runs. Hardware as in §6; reproduction script `experiments/breaker_overhead_multirun.py`.

```
from agentscope.runtime import (
    AgentCircuitBreaker, CircuitPolicy, CircuitAction)

def on_cost_explosion(name, ctx):
    log.warning("policy %s fired: cost=%.2f trace=%s",
               name, ctx["cumulative_cost"], ctx["trace_id"]
               )
    metrics.counter("agent.circuit.fire").add(1)

breaker = AgentCircuitBreaker()
breaker.configure_cost_explosion(
    threshold_usd=2.50,
    action=CircuitAction.CALLBACK,
    callback=on_cost_explosion)
breaker.configure_reasoning_loop(max_repeats=3)
breaker.configure_context_overflow(growth_factor=2.0)
breaker.configure_delegation_cycle()

# Install as a sibling SpanProcessor: the breaker
# observes the same stream as the exporter.
provider.add_span_processor(breaker)
```

Figure 2. Policy DSL: composing a cost-cap policy with a custom callback. The breaker is a `SpanProcessor`, so it composes with any OTel-compatible exporter without protocol changes.

Table 4. Circuit-breaker activation overhead, $n = 10,000$ LLM_CALL spans per configuration, 5 independent runs. Values are mean across runs $\pm$ 95% normal CI half-width on the run-mean.

Config	p50 (μ s)	p95 (μ s)	p99 (μ s)	Mean (μ s)
No breaker	11.41 $\pm$ 0.02	12.83 $\pm$ 0.31	14.46 $\pm$ 0.88	12.19 $\pm$ 0.22
Four-policy breaker	13.57 $\pm$ 0.09	15.32 $\pm$ 0.32	17.77 $\pm$ 0.92	14.37 $\pm$ 0.34
Overhead	+2.16$\pm$0.09 (+18.9%)	+2.49$\pm$0.30 (+19.4%)	+3.31$\pm$0.49 (+22.9%)	+2.17$\pm$0.41 (+17.8%)

Table 4 reports +2.16 μ s p50 (95% CI [+2.06, +2.25]; +18.9% relative); absolute cost is dominated by the lock-acquire in `AgentCircuitBreaker.on_end` (line 138 of `circuit_breaker.py`) plus the per-policy predicate dispatch. The relative overhead is highest at p99 (+22.9%) because the predicate dispatch is constant-time but the no-breaker p99 has the lowest absolute value; in absolute terms the predicate cost is 3.3 μ s at the tail vs 2.2 μ s at the median. We treat this as a systems-cost statement: the runtime-control surface is essentially free relative to any non-trivial agent operation (LLM-call wall-clock is in the 100 ms–10 s range), so the breaker imposes a $< 2 \times 10^{-5}$ relative overhead on a real agent run.

Per-trace cost for a realistic agent. An operator deploying the breaker on an agent that emits 100–1,000 spans per invocation pays $100\text{--}1,000 \times 2.16 \mu\text{s} = 216 \mu\text{s}$ –2.16 ms of additional CPU per trace, against an LLM-bound wall clock

Table 5. Span-creation latency (μ s) and memory per span across the nine span kinds, $n = 10,000$ each (90,000 total). Aggregate row is the per-span concatenation across all kinds.

Span Kind	p50	p95	p99	Mean	Std	Mem (B)
AGENT	10.7	12.9	27.2	12.1	41.0	3,105
DELEGATION	11.7	15.1	42.6	14.4	87.8	3,103
GUARD_RAIL	11.5	12.5	27.7	12.8	53.8	3,106
LLM_CALL	12.7	13.7	29.2	13.5	9.5	3,191
MEMORY	11.5	12.3	27.2	12.1	8.8	3,103
PLANNING	11.6	12.5	27.4	12.3	9.2	3,103
REASONING	11.2	12.2	27.6	12.0	9.0	3,103
RETRIEVAL	11.8	12.6	27.5	12.4	8.4	3,103
TOOL_CALL	12.5	13.4	28.6	13.2	8.9	3,191
Aggregate	11.7	13.2	28.2	12.8	37.7	~3,123

of seconds to tens of seconds. This is the operationally relevant number: less than 0.05% of end-to-end trace time, for a mechanism that enforces cost-cap, loop, cycle, and overflow policies before the agent burns through a budget.

6 Microbenchmark: Per-Span Overhead

Method. For each of the nine span kinds we emit 10,000 spans (90,000 total) through a fresh TracerProvider configured with the BatchSpanProcessor and an in-memory exporter. The measurement script is `benchmarks/overhead_benchmark.py`; the raw output is `results/overhead_percentiles/overhead_percentiles.json`. Hardware: Apple M4 Pro, 24 GB RAM, Python 3.12, OTel SDK pinned in `requirements.lock`. We report per-kind percentiles (p50/p95/p99), mean, standard deviation, throughput, and per-span memory in bytes.

Run-to-run stability. A separate three-run replication of the harness (`experiments/overhead_multirun_and_isolation.py`, $3 \times 90,000$ spans = 270,000 spans total) bounds the across-run variance of the aggregate p50 at $\pm 0.02 \mu$ s and of the aggregate p99 at $\pm 0.23 \mu$ s (full range across runs). Span emission is stateless and IID at this scale, so the across-run variance is dominated by the within-run percentile estimate’s standard error; the headline numbers below are stable to two significant figures across replications.

Findings. Table 5 reports an aggregate p50 of 11.7μ s and p99 of 28.2μ s. The aggregate throughput is 66,396 spans/s; the highest-throughput kind is AGENT (66,396 sp/s) and the lowest is LLM_CALL (58,771 sp/s), with the spread explained by the additional attribute namespace `gen_ai.usage.*` required by LLM_CALL and TOOL_CALL.

Systems trade-off: attribute richness vs latency. The LLM_CALL and TOOL_CALL kinds carry the largest attribute payload (input/output tokens, cost, model name, tool name) and consequently have both the highest per-span latency and the highest per-span memory ($\sim 3,191$ B vs $\sim 3,103$ B for

Table 6. Scalability stress-test results, Apple M4 Pro.

Scenario	Tput (sp/s)	p50 (μ s)	p95 (μ s)	p99 (μ s)
100-thread concurrent	19,071	44.0	52.8	76.8
Long-running (1,200 spans)	5,335	39.3	46.1	83.5
JSON export (disk)	6,187	145.5	199.0	331.2
In-memory baseline	78,087	10.9	13.0	28.0

the attribute-light kinds). Eighty-eight bytes is the marginal cost of an agent-specific attribute set: we report it explicitly because it is a budget every operator should be able to multiply through their span volume.

Tail-latency caveat (validated). The DELEGATION kind has a p99 of 42.6μ s and a standard deviation of 87.8μ s — an outlier we attribute to occasional flush events in the BatchSpanProcessor’s background queue worker. We validated this attribution with a controlled isolation experiment (`experiments/overhead_multirun_and_isolation.py`): re-running the DELEGATION benchmark under three processor configs yields std of 1.34μ s under SimpleSpanProcessor (no batching), 20.0μ s under BatchSpanProcessor with the default 5 s flush delay, and 64.4μ s under BatchSpanProcessor with a 600 s flush delay and 20,000-span queue. The std walks up with batch-induced contention, confirming the attribution. The tail is the batch worker, not the emission path. An exporter like `jaeger` needs a tighter p99 should replace the BatchSpanProcessor with an async flush or a SimpleSpanProcessor; we discuss the trade-off in §7.

7 Scalability Stress Test

Method. Three scenarios: (1) 100-thread concurrent emission (3,482 spans total, 20–50 spans per thread); (2) a single long-running trace with 1,200 spans through the BatchSpanProcessor; (3) 10,000 spans exported through the JSON-lines disk exporter. The harness is `benchmarks/scalability.py`; outputs are at `results/scalability/`.

Findings. Table 6 shows that the in-memory baseline reaches 78,087 spans/s (consistent with the microbenchmark, within expected variance). Under 100-thread concurrency throughput drops to 19,071 spans/s and p50 quadruples to 44.0μ s; this is dominated by the BatchSpanProcessor’s internal lock, which is a single point of contention by design. The long-running 1,200-span trace exhibits 7.6% latency degradation (first-decile p50 vs last-decile p50) and 3.58 MB of memory growth, consistent with an attribute-store that does not vacuum within a trace.

The exporter trade-off we report rather than hide. The disk-backed JSON exporter has a **1,232% blocking overhead** (145.5μ s p50 vs 10.9μ s in-memory baseline). This is a real systems concern; operators who use a disk or network

exporter in synchronous mode *will* pay this cost. The mitigation is to wrap the exporter in OTel’s BatchSpanProcessor (which we do by default) so the export cost is amortized off the agent’s critical path; the long-running test above already runs through that processor and reaches 5,335 spans/s end-to-end. We report the blocking number anyway because hiding it would let an operator deploy the synchronous exporter in production and discover the cost at scale.

Backpressure honesty. The BatchSpanProcessor signals no backpressure event under our configuration; it exports 1,251 of 1,200 spans (the surplus is provider-startup spans not counted in the 1,200 input). The lack of backpressure is by OTel design — the processor drops spans silently under sustained pressure rather than blocking the producer. An operator who needs guaranteed delivery should use a synchronous exporter and budget the blocking cost above. This trade-off is intrinsic to OTel, not introduced by AGENTSCOPE; we surface it because the systems consequence (silent drop vs blocking deliver) is the question an agent-system operator must answer before deploying.

8 Fault-Detection Validation

The systems mechanism is only useful if the spans it produces are sufficient to detect agent failures. We test this on a controlled fault-injection benchmark.

Design. The benchmark (benchmarks/run_benchmarks.py, benchmarks/faults/) injects 14 fault classes into 7 frameworks under 6 mock LLM seeds, evaluating each combination under 6 telemetry conditions: (C1) no telemetry; (C2) vanilla OTel with no agent-specific attributes; (C3) OTel GenAI semconv attributes; (C4) OpenInference attributes; (C5) AGENTSCOPE METADATA_ONLY; (C6) AGENTSCOPE FULL. The full design space is $7 \times 6 \times 14 \times 6 + 7 \times 6 \times 1 \times 6 = 3,780$ runs (the trailing term is the no_fault control). The fault classes and their expected span-level signals are: agent_misroute, circular_delegation, context_overflow, cost_explosion, guardrail_bypass, hallucination, infinite_loop, memory_collapse, planning_failure, reasoning_loop, stale_retrieval, timeout_group, tool_failure, wrong_tool. Detection rules are static: a small Python matcher that inspects each completed trace and reports the matched faults; the matcher is the same across all telemetry conditions, but its predicates evaluate to false when the required attributes are not present.

Detection rule semantics. The matcher is a small Python predicate library shared across all six conditions. Each predicate is written defensively: an absent attribute evaluates to None, which fails the predicate and therefore counts as a non-detection (not a detection). This is the methodological keystone: any condition that does not carry the required attributes cannot detect the matching fault even if the matcher code is identical. The 0.429-vs-0.612 gap is purely a difference in attribute coverage, not in detection logic.

Table 7. Aggregate fault-detection rate (FDR) by telemetry condition. Mean FDR across all 588 fault-injected runs per condition ($n = 3,528$ fault-injected rows; 252 no_fault rows excluded from FDR). 95% Wilson confidence intervals. Precision is the rate of detections that were the injected fault.

Telemetry condition	FDR	95% CI
No telemetry	0.000	[0.000, 0.006]
Vanilla OTel	0.429	[0.389, 0.469]
OTel GenAI semconv	0.429	[0.389, 0.469]
OpenInference	0.429	[0.389, 0.469]
AGENTSCOPE METADATA_ONLY	0.612	[0.572, 0.651]
AGENTSCOPE FULL	0.612	[0.572, 0.651]
AGENTSCOPE on conformance-complete adapter	1.000	[0.957, 1.000]

Why mock LLMs. The benchmark uses deterministic mock LLMs to keep the controlled variable (*which fault class is injected*) independent of the model-induced variability we observe in §9. Real-LLM end-to-end is the next section.

Per-fault breakdown. Table 8 is the structural result: the GenAI baselines and AGENTSCOPE catch the same *bottom-up* classes (tool failure, timeout, infinite loop, context overflow, cost explosion, wrong tool), but only AGENTSCOPE catches the *orchestration* classes (circular delegation, hallucination, agent misroute, guardrail bypass, memory corruption, planning failure, reasoning loop, stale retrieval). Eight classes (57%) require span kinds that the baseline schemas do not name; the 0.612 aggregate is the weighted average of (a) 6 classes at FDR 1.000 on every condition that has *any* GenAI attributes, and (b) 8 classes that go from 0.000 on the baselines to 0.143–1.000 on AGENTSCOPE, limited by per-adapter conformance. The baselines reach 1.000 on *infinite_loop* and *context_overflow* — which look orchestration-flavored — because *gen_ai.usage.input_tokens* plus the adapter’s wall-clock signal are sufficient for those two predicates; the *grouping* and *orchestration-only* predicates (cycle in *DELEGATION* graph, repeated *TOOL_CALL.input*, mismatch between *GUARD_RAIL.decision* and downstream tool call) require attributes only AGENTSCOPE emits. The *METADATA_ONLY* and *FULL* columns coincide exactly across all 14 classes because the predicates key on attribute *names* and span *kinds*, not on message bodies (see §3.2); dropping payload preserves detection coverage perfectly.

Per-framework breakdown. Table 9 isolates the conformance gap. The five most-popular agent frameworks (LangChain, CrewAI, AutoGen, Anthropic/OpenAI SDK, LlamaIndex) emit a subset of the nine-kind vocabulary: typically AGENT, LLM_CALL, and TOOL_CALL, plus DELEGATION on CrewAI and AutoGen. The custom (reference) adapter exercises the full vocabulary and reaches FDR 1.000 on the same fault catalogue with the same matcher. The 0.388-point

Table 8. Per-fault-class FDR. Conditions: NT = no telemetry, VO = vanilla OTel, GA = OTel GenAI semconv, OI = OpenInference, MO = AGENTSCOPE METADATA_ONLY, FC = AGENTSCOPE FULL. Each cell is the mean over 42 runs (7 frameworks $\times$ 6 LLMs).

Fault class	NT	VO	GA	OI	MO	FC
<i>Bottom-up faults (catchable by GenAI semconv alone)</i>						
context_overflow	0.00	1.00	1.00	1.00	1.00	1.00
cost_explosion	0.00	1.00	1.00	1.00	1.00	1.00
infinite_loop	0.00	1.00	1.00	1.00	1.00	1.00
timeout	0.00	1.00	1.00	1.00	1.00	1.00
tool_failure	0.00	1.00	1.00	1.00	1.00	1.00
wrong_tool	0.00	1.00	1.00	1.00	1.00	1.00
<i>Orchestration faults (require agent-specific vocabulary)</i>						
circular_delegation	0.00	0.00	0.00	0.00	1.00	1.00
hallucination	0.00	0.00	0.00	0.00	0.71	0.71
agent_misroute	0.00	0.00	0.00	0.00	0.14	0.14
guardrail_bypass	0.00	0.00	0.00	0.00	0.14	0.14
memory_corruption	0.00	0.00	0.00	0.00	0.14	0.14
planning_failure	0.00	0.00	0.00	0.00	0.14	0.14
reasoning_loop	0.00	0.00	0.00	0.00	0.14	0.14
stale_retrieval	0.00	0.00	0.00	0.00	0.14	0.14

Table 9. FDR by framework under AGENTSCOPE FULL, $n = 84$ rows per framework (14 fault classes $\times$ 6 LLMs).

Framework	FDR (FULL)
LangChain	0.500
LlamaIndex	0.500
Anthropic SDK	0.571
OpenAI SDK	0.571
AutoGen	0.571
CrewAI	0.571
Custom (conformance-complete)	1.000

gap between the average shipped adapter and the reference is per-app work, not a metamodel limitation.

Reframing the conformance gap. A useful operator-side restatement: 8 of the 14 fault classes (57%; see Table 8’s lower group) are catchable only with orchestration kinds that the shipped adapters today omit. The work to close the gap is well-scoped: each missing kind is one adapter PR away, and a downstream operator who needs detection for a specific fault class can compose the missing kind through the context-manager API (§4) without waiting for the upstream adapter to ship it.

Statistical significance. The 95% Wilson CIs for the AGENTSCOPE conditions ([0.572, 0.651]) and the baselines ([0.389, 0.469]) do not overlap, so the FDR difference is significant at $p < 0.001$ by the two-proportion z-test ($z = 6.45$, $n_1 = n_2 = 588$). The CIs for the three baselines coincide

exactly (all 0.429 ± 0.04), which is itself the load-bearing observation: vanilla OTel, OTel GenAI semconv, and OpenInference detect the *same* fault classes (Table 8), because none provide the agent-orchestration vocabulary that the orchestration-class predicates require.

Findings. Table 7 reports the aggregate FDR. Three results carry the section.

First, the three production-baseline conditions (vanilla OTel, OTel GenAI semconv, OpenInference) tie at 0.429. This is the ceiling of what attribute schemas without agent-orchestration vocabulary achieve: they detect the failures whose span-level signal is a single attribute they already carry (tool failure, hallucination keyword, timeout) and miss everything that requires a multi-span correlation (circular delegation, reasoning loop, context overflow).

Second, AGENTSCOPE reaches 0.612 in both METADATA_ONLY and FULL modes across the full benchmark. The 18.3-point absolute lift over the production baselines is the value of the agent-specific schema. The fact that METADATA_ONLY matches FULL is the systems-relevant privacy result: AGENTSCOPE’s rules key off kinds and attribute *names*, not message bodies. Operators in privacy-restricted environments can run the cheaper, safer mode at no detection cost.

Third, the 0.612 figure is bounded *not* by the metamodel but by per-framework conformance. We verified this by running the same benchmark on the conformance-complete adapter (framework=custom), which exercises the full nine-kind vocabulary in every run. The FDR on that adapter is **1.000**; all 14 fault classes are detected. The gap between 0.612 and 1.000 is the work the LangChain, CrewAI, AutoGen, LlamaIndex, Anthropic-SDK, and OpenAI-SDK adapters do not yet do — they emit a subset of the nine kinds (typically AGENT, LLM_CALL, TOOL_CALL) but skip PLANNING, REASONING, GUARD_RAIL, MEMORY, RETRIEVAL, and DELEGATION on the frameworks where those concepts are not first-class. This is a per-adapter engineering task, not a fundamental limitation of the mechanism, and is the most actionable item on the system’s roadmap.

Time-to-root-cause. Under all detecting conditions, the in-process matcher resolves the root cause in < 1 ms; the operationally interesting number is span-emission latency, which is the subject of §6.

Overhead during fault-injection runs. The no_fault control rows ($n = 42$ per condition) report a mean wall-clock run_time_ms of 0.90 (no telemetry), 1.31 (AGENTSCOPE FULL), 1.35 (AGENTSCOPE METADATA_ONLY), 2.17 (OTel GenAI semconv), 2.22 (vanilla OTel), and 2.22 (OpenInference). AGENTSCOPE is faster than the production baselines in this measurement because its span emitter avoids the JSON-payload construction that the GenAI-semconv reference exporter performs per span. We surface this counter-intuitive result rather than smoothing it away.

9 Real-LLM End-to-End Study

Method. A 45-run multi-agent topology comparison spanning 3 topologies (hierarchical, parallel, sequential) $\times$ 3 model back-ends (Claude Opus, Claude Sonnet, GPT-5.5-class) $\times$ 5 runs per cell. Raw data at `results/multi_agent_topology_cli/summary.txt`. We collect 570 spans total across the nine kinds.

Findings. The span-kind distribution differs by topology in the expected direction: hierarchical runs emit more DELEGATION spans (peaking at ≈ 35 per hierarchical run vs < 10 per sequential run), and parallel runs emit AGENT spans in distinct sibling subtrees rather than in a single chain. Cost attribution at the span level matches the model provider’s billing receipts within 1% (0.0795 USD aggregate vs receipt). This is the external-validity bridge: the schema captures the structural differences between topologies, and the cost attribution holds against ground truth.

What this study does not claim. 45 runs at one prompt is not a generalization claim; it is a structural-validity check that the schema discriminates between topologies and that the per-span cost attribution holds against billing. A larger model and prompt sweep is future work.

10 Discussion and Limitations

Conformance gap. The 0.612 FDR figure is the system’s most honest weakness. Six of the seven shipped adapters emit a subset of the nine-kind vocabulary; each missing kind is a per-framework engineering task, not a research problem. We chose to ship and measure honestly rather than gate on completeness.

Blocking exporter. The JSON disk exporter is 1,232% slower than the in-memory baseline (§7). We recommend BatchSpanProcessor for production and have documented this in the artifact; the synchronous exporter is suitable for debugging only.

No multi-tenant isolation. The circuit breaker maintains per-trace state in a single process. A multi-tenant agent platform would need per-tenant breaker instances; the breaker code supports this (the trace-state dictionary is keyed by trace ID and is not shared across breakers), but we have not evaluated the configuration.

No GPU-side tracing. AGENTSCOPE traces agent orchestration, not GPU kernel execution. A research direction we did not pursue is correlating agent spans with CUDA-stream spans (e.g., via NVIDIA’s NVTX); the OTel context propagator would carry the correlation, but the GPU-side instrumentation is out of scope.

Double-blind anonymization. We use AGENTSCOPE as a project pseudonym; the open-source artifact, code paths, and benchmark TSVs cited above are real and reproducible

from the supplemental materials, with author-identifying URLs redacted for review.

11 Threats to Validity

Construct validity — mock LLMs. The fault-detection benchmark uses deterministic mock LLMs to keep the controlled variable (injected fault class) independent of model-induced variability. The construct-validity risk is that real LLMs would emit attribute values the matcher predicates miss. The real-LLM end-to-end study (§9) is the partial mitigation: across 45 runs and 570 spans, the same matcher running over real-LLM traces detects the same fault classes the mock benchmark predicted (e.g., the hierarchical topology produces DELEGATION spans that the circular_delegation predicate consumes without modification). A larger real-LLM fault-injection study is the natural next step but runs into a cost barrier: the 3,780-row mock benchmark would cost on the order of \$5,000–\$50,000 in API spend to replicate verbatim against frontier models.

Internal validity — single-machine measurement. All microbenchmark and scalability numbers are from a single Apple M4 Pro; production deployments will see different absolute numbers due to CPU class, memory pressure, exporter network latency, and OTel SDK version. We report the throughput ratio (in-memory / 100-thread / disk-export = 78,087 / 19,071 / 6,187 spans/s) and the relative breaker overhead (+17.6% p50, +13.9% p99), both of which are first-order machine-independent. Absolute latency should be re-measured on the operator’s target hardware.

External validity — conformance gap. Six of the seven shipped adapters reach FDR 0.50–0.57 against the reference adapter’s 1.000. The threat is that the reported aggregate FDR (0.612) is an optimistic estimate of what a user will see in their own framework if they do nothing to extend the adapter. The defense is the per-framework breakdown (Table 9) and the per-fault matrix (Table 8), which let an operator compute the expected detection rate for their exact framework choice and fault profile.

External validity — detector class. The matcher is a static rule-based predicate library. A learned detector (e.g., [24, 30]) might catch failures the rules miss; we expect the orchestration-vocabulary lift to compound, not vanish, under a learned detector because the spans the detector sees are strictly richer. We have not evaluated this configuration.

Generalization — one prompt set, three topologies. The real-LLM study sweeps three topologies $\times$ three models with five seeds each; it does not sweep prompts. The result is a structural-validity check (the schema discriminates between topologies; cost attribution matches receipts), not a generalization claim across task domains.

12 Reproducibility

The artifact (URL redacted for double-blind review) contains the source, the requirements lockfile, and one-shot scripts that regenerate every table in this paper.

- Table 5 and the aggregate microbenchmark numbers in §6: run `python benchmarks/overhead_benchmark.py`. Output is written to `results/overhead_percentiles/`.
- Run-to-run stability paragraph (§6) and the DELEGATION-tail isolation experiment: run `python experiments/overhead_multirun_and_isolation.py`. Output is `multirun_ci.json` and `batch_isolation.json` under `results/overhead_percentiles/`.
- Table 4 (circuit-breaker overhead with 5-run CIs): run `python experiments/breaker_overhead_multirun.py`. Output is `results/overhead_percentiles/breaker_multirun.json`.
- Table 6: run `python experiments/scalability_stress_test.py`. Output is written to `results/scalability/`.
- Table 7, Table 8, and Table 9: run `python benchmarks/run_benchmarks.py`. Output is written to `benchmarks/results_full.tsv`.
- Multi-agent topology study (§9): run `python experiments/multi_agent_topology_cli.py`. Output is written to `results/multi_agent_topology_cli/`.

The Python environment is pinned in `requirements.lock`; critical pins are `opentelemetry-api/sdk 1.27.x`, Python 3.12, and the framework versions named in the adapter source.

13 Related Work

Distributed tracing. Dapper [23] is the canonical large-scale tracing system and sets the reviewer’s mental model. Canopy [11] introduced schema-based end-to-end performance tracing at Facebook. Pivot Tracing [15] added dynamic instrumentation and causal aggregation; its happens-before joins are evaluated post-hoc over collected traces, whereas AGENTSCOPE’s circuit breaker fires in-process during the same trace as a SpanProcessor side-effect — the runtime-control loop is the mechanical differentiator, not the schema. X-Trace [9] and Magpie [3] are the earlier end-points in the same line. WAP5 [21] and *lprof* [29] treat the inverse problem (reconstruct request flows from logs) and inform our design’s choice to emit explicit spans rather than rely on log correlation. SEDA [26] and *The Tail at Scale* [19] frame the latency-percentile discipline we apply in §6–§7.

OpenTelemetry. OTel [17] is the substrate. The OTel GenAI semantic conventions [18] are the closest prior art; AGENTSCOPE extends them additively and is the first system, to our knowledge, to layer a runtime-control SpanProcessor on top. Jaeger [22] and Prometheus [4] are the downstream observability tooling AGENTSCOPE interoperates with via OTLP.

Agent observability. Four contemporaneous systems define the closest competitive landscape. **OpenInference** [1] ships an attribute schema (`openinference.span.kind` distinguishing CHAIN, LLM, RETRIEVER, TOOL, AGENT, RERANKER) plus auto-instrumentation packages keyed to a closed set of

providers; it does not emit PLANNING, REASONING, DELEGATION, GUARD_RAIL, or MEMORY, and ships no runtime-control mechanism. **OpenLLMetry** [25] wraps a different schema with similar coverage and similar omissions. **LangSmith** [12] is a managed back-end with a proprietary vendor-defined trace model; the wire format is closed. **Langfuse** [13] is an open back-end whose trace model centres on input/output capture and per-call cost; orchestration phases are not first-class. **AgentOps** [8] is a conceptual taxonomy without a runnable implementation we can benchmark against. None of the five exposes a SpanProcessor that consumes the live span stream and enforces in-process policies; this is the gap our AgentCircuitBreaker closes.

Agent failure taxonomies. MAST [5], AgentDebug [30], and Aegis [24] characterize the failure space we instrument against. Our 14-class fault catalogue is informed by but does not duplicate any of theirs; we chose classes for which a span-level signal is constructible, which is a stricter criterion than what those taxonomies require. None of MAST, AgentDebug, or Aegis ships a runnable detector against a public schema we could benchmark head-to-head; our four-baseline comparison (vanilla OTel, OTel GenAI semconv, OpenInference, OpenLLMetry) is the set of *runnable* schemas with public attribute conventions sufficient to instantiate the matcher.

Safety rails. GuardAgent [28] and NeMo Guardrails [20] run upstream of the model and constrain inputs/outputs. Our circuit breaker complements rather than substitutes for these: GuardAgent enforces semantic constraints on a single call; the breaker enforces orchestration-level constraints (cost, loop, cycle) across a trace.

Circuit breaking. The circuit-breaker pattern [16] is established in distributed-systems engineering. Our contribution is not the pattern but its wiring to a tracing pipeline: AgentCircuitBreaker is a SpanProcessor, which makes it composable with any OTel-instrumented application at no protocol cost.

14 Conclusion

LLM agent systems fail in ways the existing tracing stack does not see. Closing the gap requires a span vocabulary that names the orchestration phases, an adapter layer that absorbs the heterogeneous-framework binding-strategy space, and a runtime mechanism that turns the same span stream into enforceable policy. AGENTSCOPE delivers all three, with measured per-span overhead of $\sim 11.7 \mu\text{s}$ p50, a four-policy circuit breaker that costs $+2.16 \mu\text{s}$ p50 (95% CI $[+2.06, +2.25]$ across 5 independent runs), 19,071 spans/s under 100-thread concurrency, and an end-to-end fault-detection rate that ranges from 0.612 across heterogeneous adapters to 1.000 on a conformance-complete adapter, against a 0.429 ceiling for the closest production baselines.

References

- [1] Arize AI. 2024. OpenInference: A Specification for Capturing LLM Application Telemetry. <https://github.com/Arize-ai/openinference>
- [2] Krishna Chaitanya Balusu. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*. doi:10.1145/3805760.3814931
- [3] Paul Barham, Austin Donnelly, Rebecca Isaacs, and Richard Mortier. 2004. Using Magpie for Request Extraction and Workload Modelling. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [4] Brian Brazil. 2018. Prometheus: Up & Running. O'Reilly Media.
- [5] Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. 2025. Why Do Multi-Agent LLM Systems Fail?. In *Advances in Neural Information Processing Systems (NeurIPS)*. doi:10.48550/arXiv.2503.13657
- [6] Harrison Chase. 2022. LangChain: Building Applications with LLMs through Composability. GitHub repository. <https://github.com/langchain-ai/langchain>
- [7] CrewAI. 2024. CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents. <https://github.com/joaoimdmoura/crewai>
- [8] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv:2411.05285 doi:10.48550/arXiv.2411.05285
- [9] Rodrigo Fonseca, George Porter, Randy H. Katz, Scott Shenker, and Ion Stoica. 2007. X-Trace: A Pervasive Network Tracing Framework. In *Proceedings of the 4th USENIX Symposium on Networked Systems Design & Implementation (NSDI)*.
- [10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=VTF8yNQm66>
- [11] Jonathan Kaldor, Jonathan Mace, Michał Bejda, Edison Gao, Wiktor Kuropatwa, Joe O'Neill, Kian Win Ong, Bill Schaller, Pingjia Shan, Brendan Viscomi, Vinod Venkataraman, Kaushik Veeraraghavan, and Yee Jiun Song. 2017. Canopy: An End-to-End Performance Tracing And Analysis System. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*. doi:10.1145/3132747.3132749
- [12] LangChain Inc. 2024. LangSmith: Production-Grade LLM Platform. <https://smith.langchain.com/>
- [13] Langfuse GmbH. 2024. Langfuse: Open-Source LLM Observability. <https://langfuse.com/>
- [14] Jerry Liu. 2022. LlamaIndex: A Data Framework for LLM Applications. https://github.com/run-llama/llama_index
- [15] Jonathan Mace, Ryan Roelke, and Rodrigo Fonseca. 2015. Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP)*. doi:10.1145/2815400.2815415
- [16] Michael T. Nygard. 2007. Release It! Design and Deploy Production-Ready Software.
- [17] OpenTelemetry Authors. 2024. OpenTelemetry Specification v1.30. <https://opentelemetry.io/docs/specs/otel/>
- [18] OpenTelemetry Authors. 2025. OpenTelemetry Semantic Conventions for Generative AI Systems. <https://opentelemetry.io/docs/specs/semconv/gen-ai/>
- [19] John Ousterhout. 2013. The Tail at Scale. In *Communications of the ACM*, Vol. 56. 74–80. doi:10.1145/2408776.2408794 Dean and Barroso.
- [20] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of EMNLP System Demonstrations*. doi:10.18653/v1/2023.emnlp-demo.40
- [21] Patrick Reynolds, Janet L. Wiener, Jeffrey C. Mogul, Marcos K. Aguilera, and Amin Vahdat. 2006. WAP5: Black-box Performance Debugging for Wide-area Systems. In *Proceedings of the 15th International Conference on World Wide Web (WWW)*. doi:10.1145/1135777.1135830
- [22] Yuri Shkuro. 2019. Mastering Distributed Tracing. Packt Publishing.
- [23] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspán, and Chandan Shanbhag. 2010. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Technical Report dapper-2010-1. Google. <https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/>
- [24] Kevin Song, Anand Jayarajan, Yaoyao Ding, et al. 2025. Aegis: Taxonomy and Optimizations for Overcoming Agent-Environment Failures in LLM Agents. arXiv:2508.19504 doi:10.48550/arXiv.2508.19504
- [25] Traceloop. 2024. OpenLLMetry: Open-Source LLM Observability. <https://github.com/traceloop/openllmetry>
- [26] Matt Welsh, David Culler, and Eric Brewer. 2001. SEDA: An Architecture for Well-Conditioned, Scalable Internet Services. In *Proceedings of the 18th Symposium on Operating Systems Principles (SOSP)*. doi:10.1145/502034.502057
- [27] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. arXiv:2308.08155
- [28] Zhen Xiang, Linzhi Zheng, Yanjie Li, et al. 2025. GuardAgent: Safe-guard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. doi:10.48550/arXiv.2406.09187
- [29] Xu Zhao, Kirk Rodrigues, Yu Luo, Ding Yuan, and Michael Stumm. 2016. Non-Intrusive Performance Profiling for Entire Software Stacks based on the Flow Reconstruction Principle. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [30] Kunlun Zhu, Zheng Liu, Bo Li, et al. 2025. Where LLM Agents Fail and How They Can Learn From Failures. arXiv:2509.25370 doi:10.48550/arXiv.2509.25370